Agentic Engineering

Marc Carré, October 2026

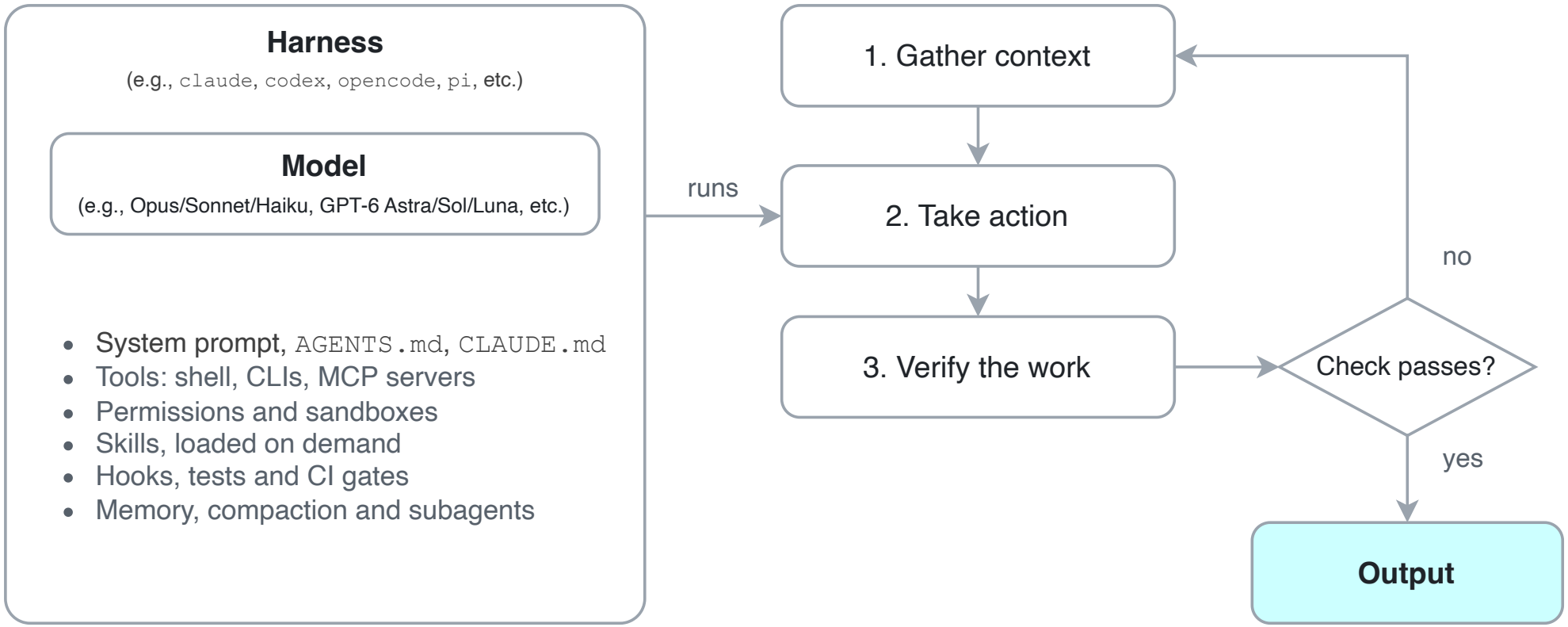
Agenda

1. Status Quo
2. Curriculum

1. Status Quo

1. Status Quo - Coding Agents

An agent is a *model* inside a *harness*, looping until a *check* passes.



⚠️: Most failures are harness failures: missing context, missing tools, or no way to verify the work.

Sources: Addy Osmani: agent harness engineering ; Anthropic: building agents with the Claude Agent SDK ; `pi` ; `opencode` ; `codex` ; `claude`

1. Status Quo - Overall Trends

Adoption of AI agents is increasing. Tooling & practices are immature and changing quickly.

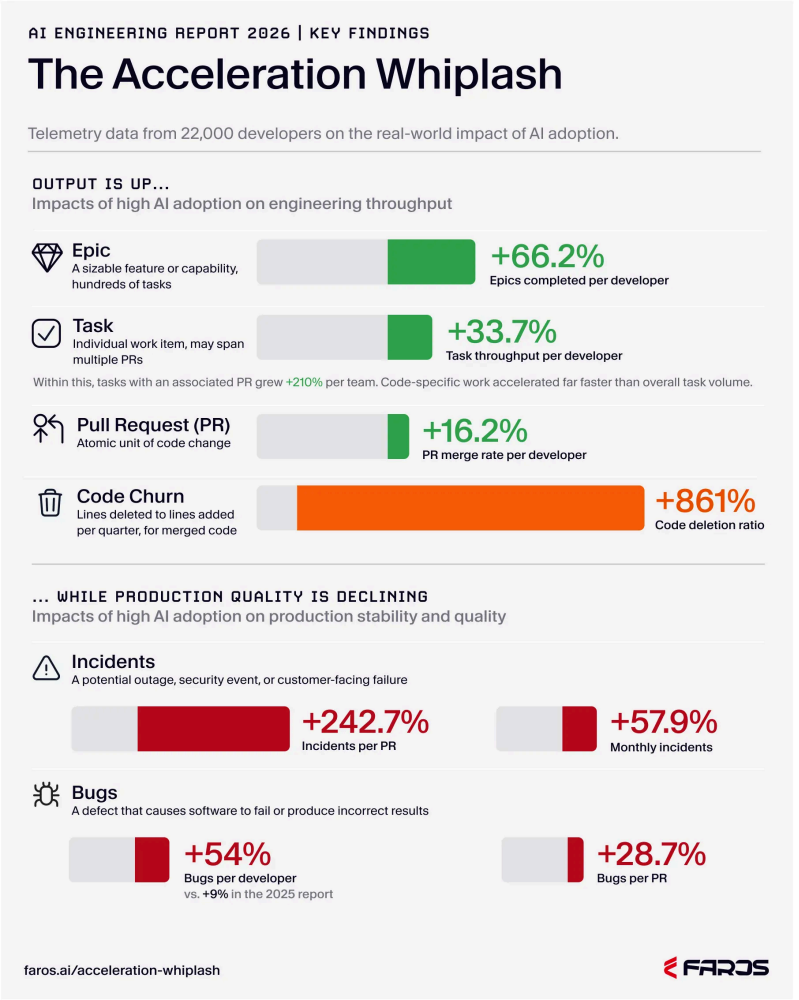
- From "*Vibe Coding*" to "*Agentic Engineering*".
- From "IDE auto-complete" to Agents in terminal, with parallel sessions, `git worktree`s, subagents, etc.
- From "*frustrated with ChatGPT*" to "*surprised by how 'good' `claude` is*".
- Still in transition & polarised: clean code → "*AI slop*", design docs → "*soup of words*", security risks, "*cognitive surrender*", burnout, etc.

#	Signal	Figure	Source
1	Engineers using AI at least 1h a day	80%	Stack Overflow 2026 survey
2	Engineers using AI agents	31% in 2025 → 59% in April 2026	Stack Overflow pulse survey
3	Trust AI output only when verifiable	48%	Stack Overflow 2026 survey
4	Claude Code market share	41% → 55%	Stack Overflow pulse survey
5	Anthropic: work done *with* AI	60%	Anthropic 2026 Agentic Coding Trends Report
6	Anthropic: work fully delegated to AI	0~20%	Anthropic 2026 Agentic Coding Trends Report
7	Experienced OSS Engineers	19% slower in early 2025; 18% faster in late 2025 (⚠️ Unreliable / Low confidence)	METR 2025 ; METR 2026

1. Status Quo - Risk #1 - Quality Gap

Writing code got faster, but *shipping* it **reliably** did NOT.

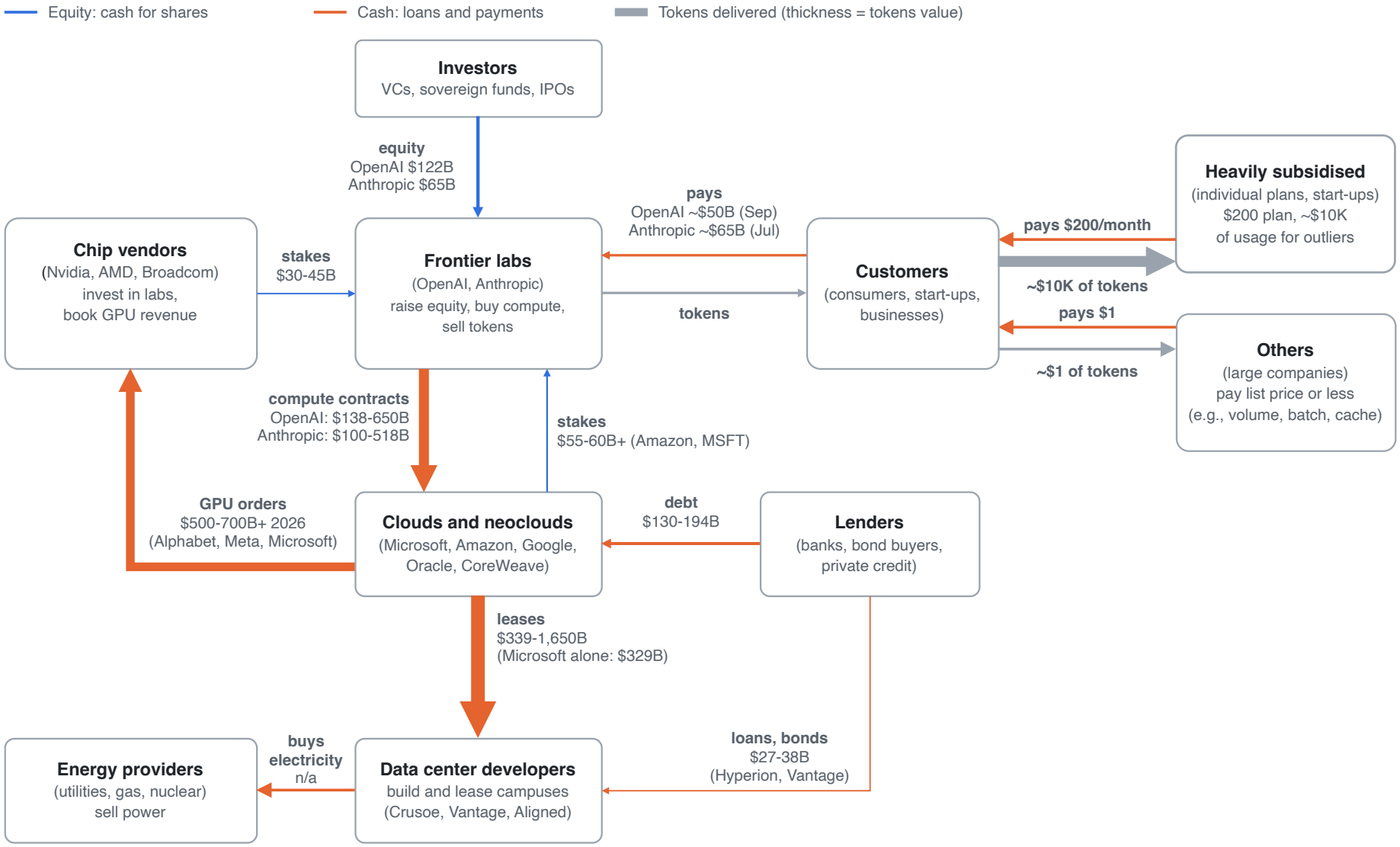
#	Signal	Figure	Source
1	Writing v.s. Debugging	79% engineers write code faster (9.8h/week), but debugging & "unpicking" AI-generated code increased (16.9h/week)	InfoQ: Undo survey
2	Understanding	35% code shipped without understanding the change	InfoQ: Undo survey
3	Hallucinated root-cause analysis	93% engineers saw at least one	InfoQ: Undo survey
4	AI-co-authored PRs	~1.7x more issues (1.75x correctness; 1.64x quality; 1.57x security; 1.42x perf.)	CodeRabbit
5	Downstream problems	+861% churn, +442% review time, +243% incidents per PR	Faros
6	Output v.s. Productivity	~4x raw output, ~12% real productivity gains	GitClear



- As of October 2026:
  - Models & harnesses are *not* smart enough (yet).
  - Tooling & processes are lacking. Everyone is learning & re-adjusting.
- DORA: "***AI is an amplifier***". It magnifies existing strengths and dysfunctions.

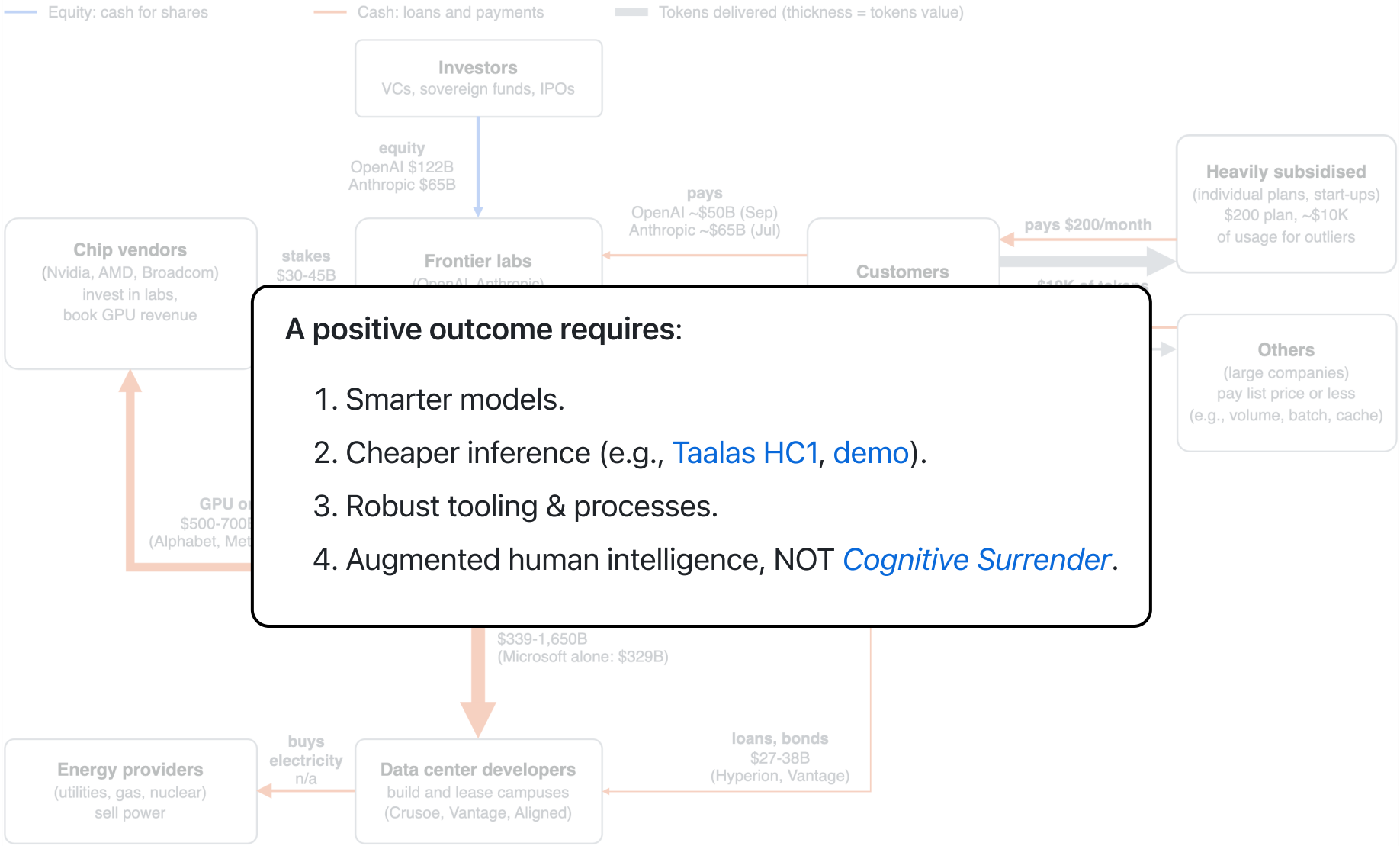
Sources: InfoQ: Undo survey (Vendor-commissioned) ; CodeRabbit (Vendor) ; DORA ; Faros (Vendor) ; GitClear (Vendor) ; Osmani (ex-Google, Anthropic).

1. Status Quo - Risk #2 - Circular Bubble or Industrial Revolution?



Sources: Company reports (Amazon 10-Q, Meta, Alphabet, Microsoft, Oracle, Anthropic, OpenAI), Bloomberg, Reuters, CNBC, Wall Street Journal, S&P, IFR, TechCrunch, Goldman Sachs, Morgan Stanley, JPMorgan, Moody's, Nikkei, Forbes, Fortune, DCD, Bank of England, BIS, Tomasz Tunguz.

1. Status Quo - Risk #2 - Circular Bubble or Industrial Revolution?



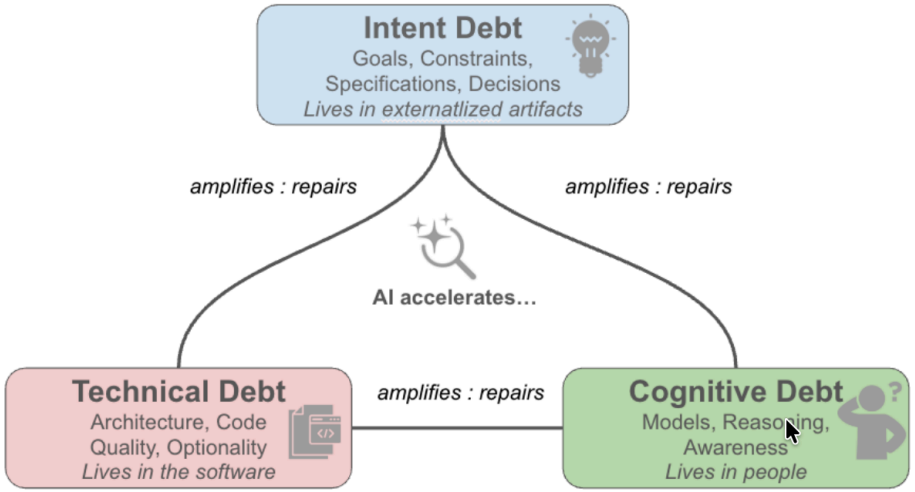
A positive outcome requires:

1. Smarter models.
2. Cheaper inference (e.g., Taalas HC1, demo).
3. Robust tooling & processes.
4. Augmented human intelligence, NOT *Cognitive Surrender*.

Sources: Company reports (Amazon 10-Q, Meta, Alphabet, Microsoft, Oracle, Anthropic, OpenAI), Bloomberg, Reuters, CNBC, Wall Street Journal, S&P, IFR, TechCrunch, Goldman Sachs, Morgan Stanley, JPMorgan, Moody's, Nikkei, Forbes, Fortune, DCD, Bank of England, BIS, Tomasz Tunguz.

1. Status Quo - Risk #3 - Cognitive Surrender & Debt - Problems

- **Cognitive Surrender**: Blindly adopting AI answers. (**73%** of AI users confidently accepted incorrect outputs.)
- **Decreased Comprehension**: AI users scored **17% lower** on comprehension (especially debugging) with zero time saved.
- In addition to **Technical Debt** (Ward Cunningham, 1993):
  - **Cognitive Debt**: Shared understanding erodes faster than it replenishes. This causes slow onboarding and burnout as AI's output is orders of magnitude above human code review abilities.
  - **Intent Debt**: Poorly documented system goals (requirements, ADRs, etc.) limit how effectively humans *and* AI can evolve the codebase.
- **Engineering Anti-Patterns**:
  - Accepting large PRs just because tests pass.
  - Applying pasted stack-trace fixes without understanding the root cause(s).
  - Comprehension debt: Pushing code nobody on the team understands.
  - etc.



Sources: Shaw and Nave (SSRN) ; Shen and Tamkin (arXiv 2601.20245) ; Storey (arXiv:2603.22106) ; Addy Osmani

1. Status Quo - Risk #3 - Cognitive Surrender & Debt - Solutions

- **Verify & Interrogate**:
  - Write down your expected answer **before** reading the AI agent's answer.
  - Ask the model to argue against its own answer.
- **Rigorous Review**:
  - Review an AI agent's PR as you would a junior engineer's PR (e.g., "tests pass" is not a review).
  - Request concrete evidence before accepting code (e.g., a trace, test run, or screenshot).
- **Scope & Skill Management**:
  - Keep changes small enough to comprehend (~100 lines).
  - Code manually weekly to stay sharp.
  - Stop using AI when you're too tired to review.
- **Optimal AI Usage**:
  - Ask for explanations alongside code. Generate code then ask follow-up questions.
  - Use AI to accelerate learning, but write the code yourself.
  - ⚠️: For humans, mistakes are where learning typically happens.

Sources: Addy Osmani ; Shen and Tamkin (arXiv 2601.20245) ; Storey (arXiv:2603.22106)

1. Status Quo - Risk #4 - Security

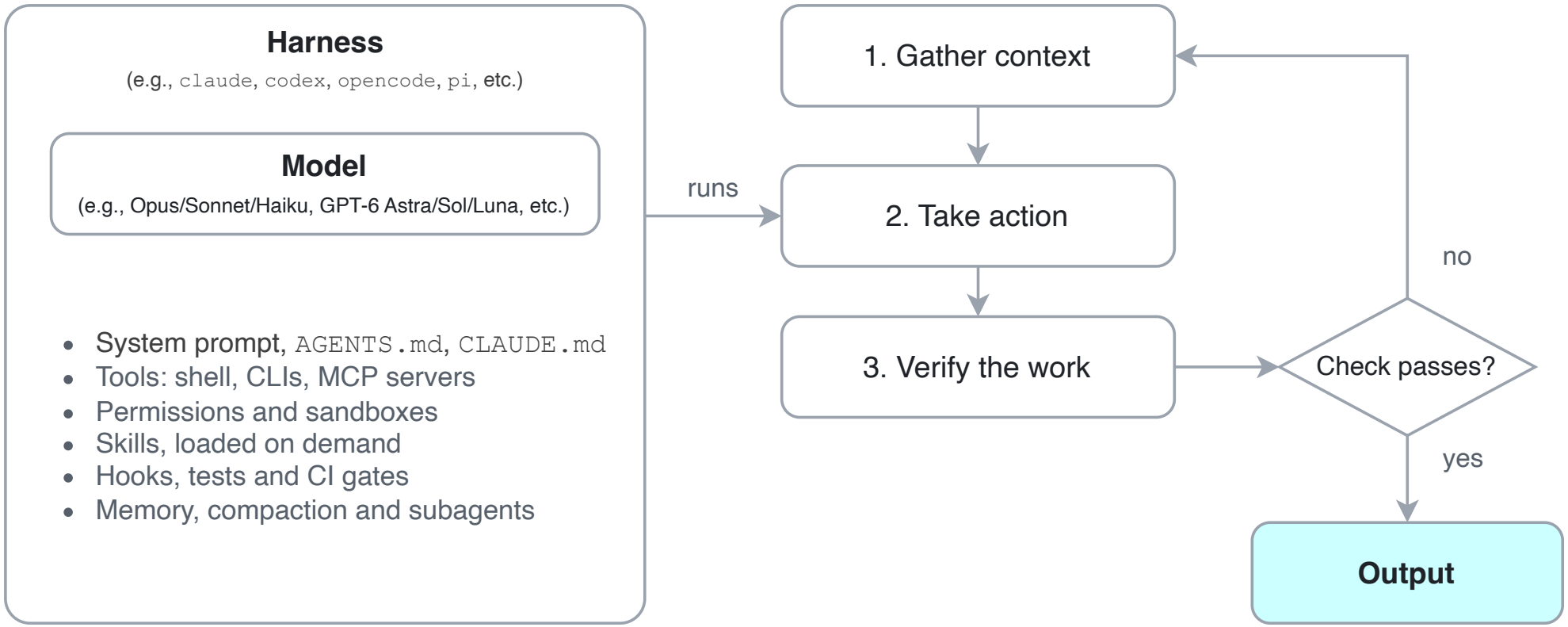
Risk Vector	Current Threat Landscape & Examples	Mitigation
Prompt Injection	The Lethal Trifecta = Private Data + Untrusted Content + Egress	Never have all three. Use sandboxes. Use secrets brokers (e.g., Automic Vault).
Sensitive Info Disclosure	AI leaking PII, credentials, or IP via outputs, logs, or APIs.	Opt-out of data training. Redact context (e.g., Microsoft Presidio, Guardrails AI). Use secrets brokers.
Supply Chain	Shai-Hulud [1][2] (500+ `npm` packages compromised) & ChainDrop [3][4] (400+ `npm` packages compromised in <4hrs).	Pin dependencies. Maintain SBOMs. Sign code. Sign inventory. Limit egress. Use short-lived OIDC credentials.
Improper Output Handling (incl. code)	AI writing code with vulnerabilities, hardcoded secrets, etc. Downstream tools or applications using unvalidated LLM output.	Use `gitleaks`. Use SAST/DAST. Use both LLM (PurpleLlama CodeShield, no-mistakes) & HITL (Human-in-the-loop) reviews. Validate output against schemas.
Excessive Agency	AI granted broad permissions, leading to data loss or financial/reputational damage.	Enforce Least Privilege (PoLP). Require Human-in-the-Loop (HITL) for destructive/high-stakes actions.
Agent Breakouts	OpenAI/Hugging Face [5][6][7] & Anthropic [8][9] evaluation breaches. Misconfigured environments.	Use sandboxes (ideally, OS-level). Do NOT rely on prompts.

See also: OWASP ; Claude Docs about Security ; Agent Control Standard (ACS)

2. Curriculum

2. Curriculum - Coding Agents (Reminder)

An agent is a *model* inside a *harness*, looping until a *check* passes.



⚠️: Most failures are harness failures: missing context, missing tools, or no way to verify the work.

Sources: Addy Osmani: agent harness engineering ; Anthropic: building agents with the Claude Agent SDK ; `pi` ; `opencode` ; `codex` ; `claude`

2. Curriculum - Overview

1 - Foundations

One session. You drive.

- **L0** First session
- **L1** Memory & `CLAUDE.md`
- **L2** Plan, build, verify
- **L3** Context & Sessions

2 - Harness

Improve the loop.

- **L4** Permissions & Sandbox
- **L5** Hooks & Gates
- **L6** Skills & Plugins
- **L7** CLIs & MCP servers
- **L8** Subagents & Review

3 - Autonomy

Human-**on**-the-loop.

- **L9** Goals & Loops
- **L10** Headless & CI

4 - Parallel

Several agents. You supervise.

- **L11** Git worktrees
- **L12** Multi-agent terminal
- **L13** Fan-out and overnight runs

5 - Economics

Tokens as a budget.

- **L14** Measure spend
- **L15** Cut tokens
- **L16** Manage costs as a team

- Assumes `claude`, for simplicity.
- Action-oriented. Usable examples.
- 1 level = 1 slide = a few **commands** with comments + the **workflow** for that level.
- Each level builds on the previous ones.

2. Curriculum - L0 First session

Install Claude Code and learn the few keys that matter.

```
curl -fsSL https://claude.ai/install.sh | bash   # install (or: brew install --cask claude-code)
cd "${REPO_PATH}" && claude                      # start a session
```

```
<Shift> + <Tab>  # inside Claude: switch modes (auto → manual → accept edits → plan)
<Esc> <Esc>      # rewind the code and/or the conversation to an earlier prompt
/clear           # start fresh before an unrelated task
```

💡 **Pro tip**: Start with questions, e.g., *"how is this class used?"*, or *"check the git history: why does this function take 15 arguments?"*. This speeds up onboarding too.

💰 **Cost tip**: Each turn re-sends the whole conversation, so `/clear` between tasks. Use `/model sonnet` for most tasks. (N.B.: default model is Opus.)

Workflow

1. Ask Claude to explain the code
2. Make one small change
3. Run it manually
4. Review code
5. Commit manually

Sources: Setup ; Interactive mode ; Checkpointing ; Boris Cherny: Mastering Claude Code in 30 minutes

2. Curriculum - L1 Memory & `CLAUDE.md`

A global `CLAUDE.md` guides *all* sessions. (Example)
A project's `CLAUDE.md` guides that project's sessions. (Example for Python projects)

```
/init         # initialise a new CLAUDE.md file with codebase documentation; ALWAYS prune/simplify it later.
/memory       # edit memory files; store personal notes in CLAUDE.local.md, git-ignored.
```

Keep *only* what Claude cannot *easily* guess from the code:

```
## Design Principles

- No ORM: always use plain SQL for users management tables. (See: docs/adr/007-no-orm.md)
```

💡 **Pro tip**: Commit a shared `CLAUDE.md` for the whole team to use the same rules. A `CLAUDE.md` in a subfolder loads only when working in that folder. Use `backpass`.

💰 **Cost tip**: `CLAUDE.md` loads on every turn: **keep it under 200 lines**. Move specialised instructions into *skills*, which load only when needed.

Workflow

1. Run `/init` → 2. Delete what Claude can guess or already gets right → 3. Add a rule only when a real mistake could repeat → 4. Review changes to it like code

Sources: Memory ; Best practices ; Addy Osmani: intent debt ; Boris Cherny: Mastering Claude Code in 30 minutes ; Kun Chen: `backpass`

2. Curriculum - L2 Plan, build, verify

Plan before code, give Claude a check it can run, and ask for evidence.

```
<Shift> + <Tab>  # into plan mode: Claude explores and proposes, no edits yet
<Ctrl> + <G>     # edit the plan in your own editor before you approve it
/clear           # after two failed corrections, start over with a better prompt
```

Write a prompt that tells Claude how to check its work:

```
users report login fails after a session timeout: <stacktrace>
check src/auth/, write a failing test that reproduces it, then fix it and show me the test output.
```

💡 **Pro tip**: Give Claude a way to check itself: linter, tests, screenshots, etc. Use `/goal`. Paste in a mock for Claude to converge faster. Use `lavish`.

💰 **Cost tip**: Name the file and function so Claude reads less. Stop early when going in the wrong direction `<Esc>` or `/rewind` : rework is the most expensive output.

Workflow

1. Explore in plan mode
2. Agree on a plan with checks at the end
3. Agent: add failing test(s) first
4. Agent: build
5. Review code

Sources: Best practices ; Permission modes: plan mode ; Boris Cherny: Mastering Claude Code in 30 minutes ; Kun Chen: `lavish`

2. Curriculum - L3 Context & Sessions

Keep the context window small, and every session easy to resume.

```
/context                              # check what fills the context window
/compact keep the plan and tests      # summarise, keeping what you specify
/rename auth-fix                      # name the session before you /clear ...
/resume                               # ... so you can come back to it later
/btw <question>                       # a side question that stays out of the history and context
```

💰 **Cost tip**: `/compact` rebuilds the prompt cache: do it between related tasks, not mid-task. To backtrack from a dead-end, run `/rewind`: it reverts the cache too.

Workflow

1. One task per session
2. Watch the context %
3. `/compact` or `/clear` at 60-80% context usage
4. Keep state in files and git

Sources: Costs ; Context window ; Checkpointing

2. Curriculum - L4 Permissions & Sandbox

Configure what Claude can/cannot do, instead of approving it click by click.

Do so in `.claude/settings.json`, **committed, for the whole team to use the same configuration.**

```
{
  "permissions": {
    "allow": ["Bash(uv run pytest*)", "Bash(git commit *)"],
    "ask":   ["Bash(git push *)"],
    "deny":  ["Read(./.env)", "Read(./.envrc)", "Read(./secrets/**)"]
  }
}
```

```
/sandbox      # isolate shell commands: files and network (macOS, Linux, WSL2)
```

💡 **Pro tip**: Configure organisation-wide rules in managed settings. E.g., **allow** reading RFCs/ADRs, **deny** unsafe/sensitive resources. Users cannot override these rules.

Workflow

1. Note the prompts you approve
2. Safe & frequent: `allow`; risky: `ask`; secrets: `deny`
3. Commit the file
4. Turn on `/sandbox`

Sources: Permissions ; Sandboxing ; Boris Cherny: Mastering Claude Code in 30 minutes

2. Curriculum - L5 Hooks & Gates

For rules to always follow (e.g., "*format every edit*", "*check every commit*", etc.):

- **add `hooks` in `.claude/settings.json`**
- **add `git` pre-commit hooks** to enforce it for *both* AI and humans, e.g., with `prek`.

```
{ "hooks": {
    "PostToolUse": [{
      "matcher": "Edit|Write",
      "hooks": [{
        "type": "command",
        "command": "jq -r '.tool_input.file_path' | xargs uv run ruff format"
      }]
    }]
}}
```

```
uv tool install prek && prek install   # format, lint, type-check and test before every commit
```

💰 **Cost tip**: Filter test output with a hook and keep only the `FAIL` and `ERROR` lines, so the full log never enters the context.

Workflow

1. Claude repeats a mistake → 2. Can a tool catch it? → 3. Add a hook, lint rule, and CI check → 4. Review logic and tests, not style

Sources: Hooks guide ; `prek`

2. Curriculum - L6 Skills & Plugins

Turn a prompt you keep retyping into a slash command, e.g.:

```
$ cat .claude/skills/fix-issue/SKILL.md

---
name: fix-issue
description: Fix a GitHub issue end to end, given its number.
disable-model-invocation: true
---
Fix issue $ARGUMENTS: reproduce it with a failing test, fix it, show the test output.
```

```
/fix-issue 1234                                                  # inside Claude: run your skill

claude plugin install mattpocock-skills@claude-plugins-official  # or install a proven pack (warning: security risk! 🔐)
claude plugin install agent-skills@addy-agent-skills
```

💡 **Pro tip**: Avoid mass-installing skills: they bloat context and degrade performance (arXiv:2602.20867 ; arXiv:2608.11888). Build your own with `/skill-creator`.

💰 **Cost tip**: For typed and type-annotated languages, install a code intelligence (LSP) plugin: Claude jumps to definitions instead of reading and searching files.

Workflow

1. Typed it several times? → 2. Make it a skill → 3. Test it in a fresh session → 4. Delete skills that overlap

Sources: Skills ; mattpocock/skills ; addyosmani/agent-skills ; Code Intelligence ; Skill Creator ; SkillsBench ; Skills invocation benchmark

2. Curriculum - L7 CLIs & MCP servers

Connect the systems you work in, with as few tools as possible.

```
gh auth login                                                    # CLIs first: no context cost

claude mcp add --transport http notion https://mcp.notion.com/mcp  # add a server
claude mcp list                                                  # check that it connects
```

```
/mcp          # inside Claude: disable the servers you don't use
```

💡 **Pro tip**: Tell Claude to run `<your-cli> --help` to learn about CLI tools, then note it in `CLAUDE.md`. Commit `.mcp.json` so the whole team gets the same servers.

💰 **Cost tip**: Prefer CLIs to MCP servers. CLIs use less context: MCP servers load their tool definitions into every session.

Workflow

1. Use a CLI if one exists
2. → Add MCP if no CLI exists
3. → Compare `/context` before and after
4. → Prefer read-only permissions

Sources: MCP ; Costs ; Model Context Protocol ; Anthropic: Code execution with MCP ; AXI ; Kun Chen: GitHub CLI vs MCP vs tool search vs code mode

2. Curriculum - L8 Subagents & Review

Never let the writer be the judge: review in a fresh context.

```
/code-review        # review your branch in a fresh context, in the background
/security-review    # a security pass against the default branch
/simplify           # cleanup only: reuse, simplification, efficiency

use a subagent to run the tests and report only the failures   # keeps noise out
```

💰 **Cost tip**: Set `model: haiku` on subagents that run tests or read logs: only their summary comes back to the main context.

💡 **Pro tip**: Run CI locally with `act`. Run `git push no-mistakes`: an agentic pipeline reviews, lints, and tests in a disposable worktree, then opens the PR.

Workflow

1. Claude writes the change
2. Claude reviews its output in a fresh context
3. You triage every finding
4. You write the PR summary

Sources: Code review ; Subagents ; Addy Osmani: agentic code review ; claude-code-security-review ; no-mistakes

2. Curriculum - L9 Goals & Loops

Let Claude keep working until a *measurable* condition becomes true.

```
# work until the condition holds; a separate model checks it after every turn
/goal all tests in test/auth pass; no other test file changes; stop after 20 turns

# repeat a prompt on a schedule
/loop 5m check if the deployment finished and tell me what happened
```

```
claude --permission-mode auto   # no prompt per step (the default since v2.1.283)
```

💰 **Cost tip**: Always cap the turns. Stop idle `/loop` and scheduled tasks: each run re-sends the whole context.

Workflow

1. Write a measurable "done" condition
2. Run in auto mode with the sandbox
3. Come back to evidence
4. Review & Merge

Sources: Goal ; Scheduled tasks ; Addy Osmani: practical loop engineering

2. Curriculum - L10 Headless & CI

Run Claude from scripts and pipelines, with exact permissions and budgets.

```
claude -p "summarise this branch" --output-format json | jq -r .result           # one-shot, scriptable

claude -p "fix the failing test in auth.test.ts" \
  --permission-mode dontAsk \
  --allowedTools "Read,Edit,Bash(npm test *)" --max-turns 10 --max-budget-usd 2   # permissions and costs capped
```

```
/install-github-app   # inside Claude: answer @claude in GitHub issues and PRs
```

💡 **Pro tip**: Pipe into `claude -p` like any Unix tool, e.g., `cat app.log | claude -p "what is unusual here?"`. Anthropic uses it in CI and incident response.

💰 **Cost tip**: Use the cheapest model that passes your checks in each job, e.g. `--model sonnet`. Keep Opus for the hard problems.

Workflow

1. Prototype the prompt interactively
2. Configure tools. Cap permissions & costs.
3. Pilot on 2–3 inputs
4. Use in scripts and CI
5. Human approval gate

Sources: Headless ; GitHub Actions ; Boris Cherny: Mastering Claude Code in 30 minutes

2. Curriculum - L11 Git worktrees

One task, one branch, one worktree: sessions never collide.

```
echo '.claude/worktrees/' >> .gitignore   # keep worktrees out of git
printf '.env\n' > .worktreeinclude        # gitignored files to copy into each worktree

claude -w feat-foobar                     # a session in its own worktree and branch
git worktree list                         # list all worktrees; remove merged ones
```

💡 **Pro tip:** `treehouse` keeps a pool of reusable worktrees, so each agent starts fast with dependencies and build cache intact.

Workflow

1. Split work into tasks → 2. One worktree and PR per task → 3. Review & merge each PR → 4. Remove merged worktrees

Sources: Worktrees ; `git worktree` ; `treehouse`

2. Curriculum - L12 Multi-agent terminal

Watch three or four agents at once, and see which one needs you.

```
# herdr: a terminal multiplexer that shows which agent is blocked
curl -fsSL https://herdr.dev/install.sh | sh && herdr integration install claude

# cmux on macOS: native panes and agent notifications; pair it with claude -w
brew tap manaflow-ai/cmux && brew install --cask cmux

# or good ol' tmux: one Claude per pane, each in its own worktree
tmux new-session -d -s agents 'claude -w a1' \; split-window 'claude -w a2' \; attach
```

💡 **Pro tip**: `firstmate` : talk to one agent, and it runs a crew of agents in their own worktrees and returns finished PRs.

💰 **Cost tip**: Track each agent's spend: `/statusline show context percentage with a progress bar, session cost, and duration` .

Workflow

1. Start with two agents
2. Give each agent a task and "done" condition
3. Unblock waiting agents
4. Review one PR at a time

Sources: `herdr` docs ; `cmux` docs ; Addy Osmani: your parallel agent limit ; `firstmate` ; Kun Chen: `firstmate` note

2. Curriculum - L13 Fan-out and overnight runs

Hand over large work that splits cleanly, and let a pipeline review it first.

```
/batch migrate @src from Python to Rust          # 5–30 units, one worktree each
```

```
no-mistakes init && git push no-mistakes         # review, lint and test, then open PR

npm install -g gnhf                              # overnight loop: commits on success, rolls back on failure
gnhf "reduce complexity without changing behaviour" --max-iterations 10 --max-tokens 5000000
```

💰 **Cost tip**: Fan-out multiplies tokens: agent teams use about 7x the tokens, and dynamic workflows use many too. Keep agent teams small. Set token caps.

Workflow

1. Check the plan before proceeding
2. → Pilot 2-3 units
3. → Let the pipeline review first
4. → Review risky changes + a sample of the rest

Sources: Commands: /batch ; `gnhf` ; `no-mistakes`

2. Curriculum - L14 Measure spend

Know what each session, task and pull request costs.

```
/usage                                  # inside Claude: this session's cost, cache hits and misses
```

```
npx ccusage@latest claude daily                                  # cost per day, from local logs
npx -y ccstatusline@latest                                       # cost and context % in your status line
claude -p "<task>" --output-format json | jq .total_cost_usd     # cost of scripted runs
```

💡 **Pro tip**: Actions that invalidate the cache: switch model, change effort, add/remove MCP server, add/remove plugin, `/compact`, upgrade Claude Code.

💰 **Cost tip**: The status line cost is an estimate at list price, and resets on `/clear`. Show the cache hit rate (`prompt_cache.hit_ratio`).

Workflow

1. Watch the status line
2. Check `ccusage` weekly
3. Track cost per merged PR
4. Review expensive outliers
5. Improve practices & process

Sources: Costs ; Prompt caching ; `ccusage` ; `ccstatusline`

2. Curriculum - L15 Cut tokens

Use the cheapest model, lowest effort, simplest setup that succeeds.
Only use better model, higher effort, etc. when it fails.

```
/model sonnet     # choose the model per task, at the start of the task
/effort low       # lower effort first: it keeps the prompt cache
/clear            # clear between tasks: the whole history is re-sent every turn
```

```
export CLAUDE_CODE_SUBAGENT_MODEL=haiku   # cheaper subagents

brew install rtk && rtk init -g           # compact shell output; measure before and after
```

💡 **Pro tips**: Cut tokens in/out. **In**: `rtk` compacts shell output; `headroom` compresses tool output, logs, and files. **Out**: `caveman` shortens replies.

💰 **Cost tip**: A model switch re-reads the whole history (`opusplan` switches at each plan toggle). Use Sonnet with `/advisor opus`. Avoid fast mode: 2x Opus costs.

Workflow

1. Measure a baseline → 2. Trim `CLAUDE.md`, MCP servers, skills, etc. → 3. Use cheaper model, low effort, cheaper subagents → 4. Measure again. Keep what helps. Repeat

Sources: Costs ; Model configuration ; `rtk` ; `headroom` ; `caveman` ; Tejas Chopra (Netflix): Headroom

2. Curriculum - L16 Manage costs as a team

Dashboards and budgets for the whole team (not tried yet!):

```
git clone https://github.com/anthropics/claude-code-monitoring-guide
cd claude-code-monitoring-guide && docker-compose up -d                    # collector, Prometheus, Grafana :3000

# send each session's metrics to the collector (for a team, put these in managed settings)
export CLAUDE_CODE_ENABLE_TELEMETRY=1 \
       OTEL_METRICS_EXPORTER=otlp \
       OTEL_EXPORTER_OTLP_PROTOCOL=grpc \
       OTEL_EXPORTER_OTLP_ENDPOINT=http://localhost:4317
```

💰 **Cost tip**: Push a cheaper default (`"model": "sonnet"`), `availableModels` and spend limits to everyone through managed settings.

Workflow

1. Create a pilot → 2. Add telemetry to managed settings → 3. Set budgets as guardrails → 4. Review cost per merged PR monthly

Sources: Monitoring ; Monitoring guide

2. Curriculum - Mindset

Working with AI Agents may require a *mindset shift*:

- We *all* become Engineering Managers of "*brilliant interns*". (No offense to interns. 🙏 No offense to AI. 🙏 No offense to managers. 🙏)
- As (Traditional *and* Agentic) Engineering Managers, we need to:
  - improve the **practices** for our "teams" to perform well,
  - remove **bottlenecks** in the current process and continuously improve it.
- The same practices that make great engineering teams *also* apply to agentic engineering teams:
  - RFCs, ADRs, etc. to capture technical **decisions** and the **context** around them,
  - CICD, tests, linting, type checking, pre-commit hooks, etc. -- for **short feedback loops**.

	Software Engineering	Engineering Management	Agentic Engineering
Language	Programming language (formal)	Natural language (ambiguous)	Natural language (ambiguous)
Behaviour	Deterministic	Non-deterministic	Non-deterministic
Input	Requirements, context	Vision, requirements, context	Requirements and context, written for agents
Quality check	Tests	Clear goals	Verifiable goals: tests, checks
Review	Code review	Product review, QA	Code, plans and evidence; a human owns every merge
Process	Low-level (BDD, TDD)	High-level (SDLC)	High-level (SDLC); agents run TDD

Next steps

1. Try and master each step
2. Share learned lessons
3. Consolidate practices
4. Accelerate *without* sacrificing understanding & quality
5. Measure team costs and performance (DORA metrics)

Recommendations:

Start small. Measure. Improve.
Keep using your brain.
Keep quality high.
Keep a human on every merge (at least for now).

Thank you! 🙏

Questions?